



LangGraph



[Resources](#)



[Definitions](#)

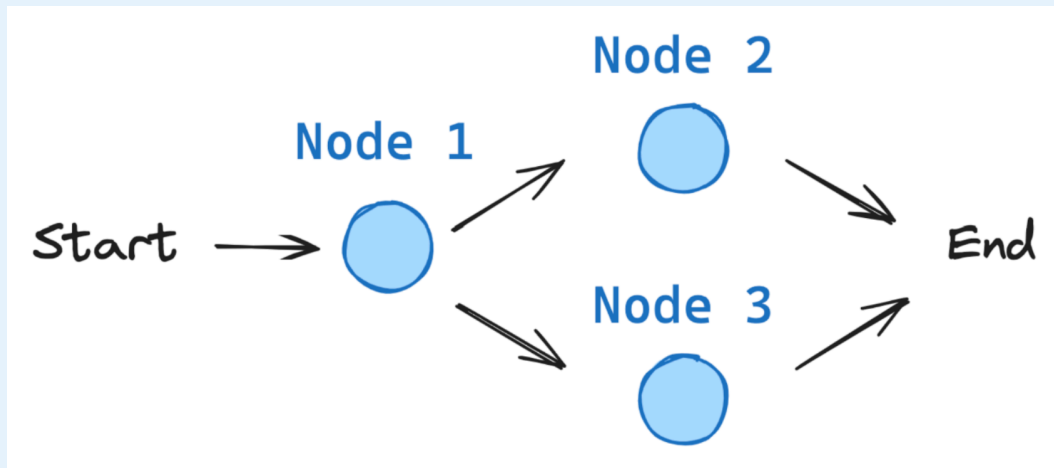


[LangGraph Deep Agents](#)

Overview

LangGraph is a framework for agent orchaestration that can:

- Control the flow of an agent between steps (nodes)
- Make long-running execution robust to failures
- Incoporate human-in-the-loop
- Provide short-term and long-term memory
- Deploy production ready agents with provided UIs
- Work with LangChain or any other frameworks



LangGraph runs on super-steps meaning node 2 and node 3 are in the same step and run in parallel...

Creating a Graph

1. Define tools & models

▼ Example

```
from langchain.tools import tool
from langchain.chat_models import init_chat_model
```

```
model = init_chat_model(
    "claude-sonnet-4-5-20250929",
    temperature=0
)
```

```
# Define tools
```

```
@tool
```

```
def multiply(a: int, b: int) → int:
    """Multiply `a` and `b`.
```

```
    Args:
```

```
        a: First int
```

```
        b: Second int
```

```
    """
```

```
    return a * b
```

```
@tool
```

```
def add(a: int, b: int) → int:
    """Adds `a` and `b`.
```

```
    Args:
```

```
        a: First int
```

```
        b: Second int
```

```
    """
```

```

    return a + b

@tool
def divide(a: int, b: int) → float:
    """Divide `a` and `b`.
```

Args:

```

    a: First int
    b: Second int
    """
    return a / b

# Augment the LLM with tools
tools = [add, multiply, divide]
tools_by_name = {tool.name: tool for tool in tools}
model_with_tools = model.bind_tools(tools)

```

2. Define `state` schema

- State is the data which our graphs carries through the graph. Each execution of the graph starts with a fresh State instance.
- Here we also define `reducers` which dictate how state is updated

▼ Example

- The `Annotated` type with `operator.add` ensures that new messages are appended to the existing list rather than replacing it.

```

from langchain.messages import AnyMessage
from typing_extensions import TypedDict, Annotated
import operator

class MessagesState(TypedDict):

```

```
messages: Annotated[list[AnyMessage], operator.add]
llm_calls: int
```

3. Define **nodes**

Sub-Graphs

Nodes & Tool Nodes

- Nodes are functions that make changes to state via calling LLMs or logic

▼ **Example**

```
from langchain.messages import SystemMessage
def llm_call(state: dict):
    """LLM decides whether to call a tool or not"""

    return {
        "messages": [
            model_with_tools.invoke(
                [
                    SystemMessage(
                        content="You are a helpful assistant tasked with performing arithmetic on a set of inputs."
                    )
                ]
            + state["messages"]
        ),
        "llm_calls": state.get('llm_calls', 0) + 1
    }
```

```
from langchain.messages import ToolMessage
def tool_node(state: dict):
```

```

        """Performs the tool call"""

        result = []
        for tool_call in state["messages"][-1].tool_calls:
            tool = tools_by_name[tool_call["name"]]
            observation = tool.invoke(tool_call["args"])
            result.append(ToolMessage(content=observation, tool_call_id=
            tool_call["id"]))
        return {"messages": result}

```

4. Define **conditional edges**

- Conditional edges are functions that choose the next node based on something

▼ Example

```

from typing import Literal
from langgraph.graph import StateGraph, START, END

def should_continue(state: MessagesState) → Literal["tool_node",
END]:
    """Decide if we should continue the loop or stop based upon w
hether the LLM made a tool call"""

    messages = state["messages"]
    last_message = messages[-1]

    # If the LLM makes a tool call, then perform an action
    if last_message.tool_calls:
        return "tool_node"

    # Otherwise, we stop (reply to the user)
    return END

```

5. Build agent

- Here we connect the nodes with conditional edges and edges to form a graph

▼ Example

```
# Build workflow
agent_builder = StateGraph(MessagesState)

# Add nodes
agent_builder.add_node("llm_call", llm_call)
agent_builder.add_node("tool_node", tool_node)

# Add edges to connect nodes
agent_builder.add_edge(START, "llm_call")
agent_builder.add_conditional_edges(
    "llm_call",
    should_continue,
    ["tool_node", END]
)
agent_builder.add_edge("tool_node", "llm_call")
```

6. Compile

Memory.

- Here we take our graph build blueprint and make it into a executable graph
- We can provide checkpointing (short term-memory) and memory stores (long-term memory) here

▼ Example

```
# Compile the agent (no memory)
agent = agent_builder.compile()
```

7. Run the graph

Running a Graph

- We can run the graph from a .ipynb notebook, deploy it locally or deploy it on external servers

▼ **Example**

```
# Run the graph in a .ipynb cell
from langchain.messages import HumanMessage
messages = [HumanMessage(content="Add 3 and 4.")]
messages = agent.invoke({"messages": messages})
for m in messages["messages"]:
    m.pretty_print()
```

LangGraph Code Examples

Aa Name

[Prompt Chaining](#)

[Parallelisation](#)

[Routing](#)

[Orchestrator-Worker](#)

[Evaluator-Optimiser](#)

[ReACT Agent](#)

Interrupts

Stop the graph at a certain point, letting us provide input and resume the graph from there...

Interrupts Under the Hood

When you call `interrupt`, here's what happens:

1. **Graph execution gets suspended** at the exact point where `interrupt` is called
2. **State is saved** using the checkpointer so execution can be resumed later, In production, this should be a persistent checkpointer (e.g. backed by a database)
3. **Value is returned** to the caller under `__interrupt__`; it can be any JSON-serializable value (string, object, array, etc.)
4. **Graph waits indefinitely** until you resume execution with a response
5. **Response is passed back** into the node when you resume, becoming the return value of the `interrupt()` call

Implementation

1. Ensure checkpointing is setup & thread is specified
2. Set a variable equal to `interrupt()` where you want to pause (i.e. in a node)
 - You can specify a JSON structure inside `interrupt` to provide details on what is needed that we can i.e. display in a UI (though this is just info, not enforced)
 - When we restart the graph we will pass back a single entry or JSON object which becomes this variable
3. `Invoke` the graph and it will stop at the interrupt
4. **Resume the graph by running** `invoke(Command(resume=<your_json>), config=config)`
 - Whatever single entry or JSON we give to resume will become the interrupted variable

▼ Demo 1: User Approve or Reject

```
from typing import Literal, Optional, TypedDict

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

class ApprovalState(TypedDict):
    action_details: str
    status: Optional[Literal["pending", "approved", "rejected"]]

def approval_node(state: ApprovalState) → Command[Literal["proceed", "cancel"]]:
    # Expose details so the caller can render them in a UI
    decision = interrupt({
```

```

        "question": "Approve this action?",
        "details": state["action_details"],
    })

    # Route to the appropriate node after resume
    return Command(goto="proceed" if decision else "cancel")

def proceed_node(state: ApprovalState):
    return {"status": "approved"}

def cancel_node(state: ApprovalState):
    return {"status": "rejected"}

builder = StateGraph(ApprovalState)
builder.add_node("approval", approval_node)
builder.add_node("proceed", proceed_node)
builder.add_node("cancel", cancel_node)
builder.add_edge(START, "approval")
builder.add_edge("proceed", END)
builder.add_edge("cancel", END)

# Use a more durable checkpointer in production
checkpointer = MemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "approval-123"}}
initial = graph.invoke(
    {"action_details": "Transfer $500", "status": "pending"},
    config=config,
)
print(initial["__interrupt__"]) # → [Interrupt(value={'question': ...,
'details': ...})]

```

```
# Resume with the decision; True routes to proceed, False to cancel
resumed = graph.invoke(Command(resume=True), config=config)
print(resumed["status"]) # → "approved"
```

▼ Demo 2: User Review & Edit State

```
import sqlite3
from typing import TypedDict

from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

class ReviewState(TypedDict):
    generated_text: str

def review_node(state: ReviewState):
    # Ask a reviewer to edit the generated content
    updated = interrupt({
        "instruction": "Review and edit this content",
        "content": state["generated_text"],
    })
    return {"generated_text": updated}

builder = StateGraph(ReviewState)
builder.add_node("review", review_node)
builder.add_edge(START, "review")
builder.add_edge("review", END)

checkpointer = MemorySaver()
```

```

graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "review-42"}}
initial = graph.invoke({"generated_text": "Initial draft"}, config=config)
print(initial["__interrupt__"]) # → [Interrupt(value={'instruction': ..., 'content': ...})]

# Resume with the edited text from the reviewer
final_state = graph.invoke(
    Command(resume="Improved draft after review"),
    config=config,
)
print(final_state["generated_text"]) # → "Improved draft after review"

```

▼ Demo 3: Tool Approval

- We can place an interrupt inside a tool

```

import sqlite3
from typing import TypedDict

from langchain.tools import tool
from langchain_anthropic import ChatAnthropic
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

class AgentState(TypedDict):
    messages: list[dict]

@tool
def send_email(to: str, subject: str, body: str):

```

```

"""Send an email to a recipient."""

# Pause before sending; payload surfaces in result["__interrupt
__"]
response = interrupt({
    "action": "send_email",
    "to": to,
    "subject": subject,
    "body": body,
    "message": "Approve sending this email?",
})

if response.get("action") == "approve":
    final_to = response.get("to", to)
    final_subject = response.get("subject", subject)
    final_body = response.get("body", body)

    # Actually send the email (your implementation here)
    print(f"[send_email] to={final_to} subject={final_subject} bod
y={final_body}")
    return f"Email sent to {final_to}"

return "Email cancelled by user"

model = ChatAnthropic(model="claude-sonnet-4-5-20250929").
bind_tools([send_email])

def agent_node(state: AgentState):
    # LLM may decide to call the tool; interrupt pauses before send
ing
    result = model.invoke(state["messages"])
    return {"messages": state["messages"] + [result]}

```

```

builder = StateGraph(AgentState)
builder.add_node("agent", agent_node)
builder.add_edge(START, "agent")
builder.add_edge("agent", END)

checkpointer = SqliteSaver(sqlite3.connect("tool-approval.db"))
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "email-workflow"}}
initial = graph.invoke(
    {
        "messages": [
            {"role": "user", "content": "Send an email to alice@example.com about the meeting"}
        ]
    },
    config=config,
)
print(initial["__interrupt__"]) # → [Interrupt(value={'action': 'send_email', ...})]

# Resume with approval and optionally edited arguments
resumed = graph.invoke(
    Command(resume={"action": "approve", "subject": "Updated subject"}),
    config=config,
)
print(resumed["messages"][-1]) # → Tool result returned by send_email

```

▼ Demo 4: User Input & Validate it

- Sometimes you need to validate input from humans and ask again if it's invalid

```

import sqlite3
from typing import TypedDict

from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command, interrupt

class FormState(TypedDict):
    age: int | None

def get_age_node(state: FormState):
    prompt = "What is your age?"

    while True:
        answer = interrupt(prompt) # payload surfaces in result["__interrupt__"]

        if isinstance(answer, int) and answer > 0:
            return {"age": answer}

        prompt = f"'{answer}' is not a valid age. Please enter a positive number."

builder = StateGraph(FormState)
builder.add_node("collect_age", get_age_node)
builder.add_edge(START, "collect_age")
builder.add_edge("collect_age", END)

checkpointer = SqliteSaver(sqlite3.connect("forms.db"))
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "form-1"}}

```

```
first = graph.invoke({"age": None}, config=config)
print(first["__interrupt__"]) # → [Interrupt(value='What is your age?', ...)]

# Provide invalid data; the node re-prompts
retry = graph.invoke(Command(resume="thirty"), config=config)
print(retry["__interrupt__"]) # → [Interrupt(value="'thirty' is not a valid age...", ...)]

# Provide valid data; loop exits and state updates
final = graph.invoke(Command(resume=30), config=config)
print(final["age"]) # → 30
```

Debugging Interrupts

- There are a special type of interrupts we can use for debugging nodes
- We specify them in `compile` or in LangSmith Studio

```
graph = builder.compile(  
    interrupt_before=["node_a"],  
    interrupt_after=["node_b", "node_c"],  
    checkpoint=checkpoint,  
)
```

```
# Pass a thread ID to the graph  
config = {  
    "configurable": {  
        "thread_id": "some_thread"  
    }  
}
```

```
# Run the graph until the breakpoint  
graph.invoke(inputs, config=config)
```

```
# Resume the graph  
graph.invoke(None, config=config)
```



- Do NOT put `interrupts` in try / except blocks
- Do NOT let the order which we flow through `interrupts` change if multiple `interrupts` in one node

Testing & Tracing

Tracing & Evals

- Use LangSmith

Testing

We may want to make pytest tests for nodes. Use Pytest:

▼ Demo of Full Testing

```
import pytest

from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import MemorySaver

def create_graph() → StateGraph:
    class MyState(TypedDict):
        my_key: str

    graph = StateGraph(MyState)
    graph.add_node("node1", lambda state: {"my_key": "hello from node1"})
    graph.add_node("node2", lambda state: {"my_key": "hello from node2"})
    graph.add_edge(START, "node1")
    graph.add_edge("node1", "node2")
    graph.add_edge("node2", END)
    return graph

def test_individual_node_execution() → None:
    # Will be ignored in this example
    checkpointer = MemorySaver()
    graph = create_graph()
    compiled_graph = graph.compile(checkpointer=checkpointer)
    # Only invoke node 1
    result = compiled_graph.nodes["node1"].invoke(
        {"my_key": "initial_value"},
```

```
)  
assert result["my_key"] == "hello from node1"
```

▼ Demo of Partial Execution testing

- So we can test just what is happening between 2 nodes for example

```
import pytest  
  
from typing_extensions import TypedDict  
from langgraph.graph import StateGraph, START, END  
from langgraph.checkpoint.memory import MemorySaver  
  
def create_graph() → StateGraph:  
    class MyState(TypedDict):  
        my_key: str  
  
        graph = StateGraph(MyState)  
        graph.add_node("node1", lambda state: {"my_key": "hello from  
node1"})  
        graph.add_node("node2", lambda state: {"my_key": "hello from  
node2"})  
        graph.add_node("node3", lambda state: {"my_key": "hello from  
node3"})  
        graph.add_node("node4", lambda state: {"my_key": "hello from  
node4"})  
        graph.add_edge(START, "node1")  
        graph.add_edge("node1", "node2")  
        graph.add_edge("node2", "node3")  
        graph.add_edge("node3", "node4")  
        graph.add_edge("node4", END)  
        return graph  
  
def test_partial_execution_from_node2_to_node3() → None:  
    checkpointer = MemorySaver()
```

```

graph = create_graph()
compiled_graph = graph.compile(checkpointer=checkpointer)
compiled_graph.update_state(
    config={
        "configurable": {
            "thread_id": "1"
        }
    },
    # The state passed into node 2 - simulating the state at
    # the end of node 1
    values={"my_key": "initial_value"},
    # Update saved state as if it came from node 1
    # Execution will resume at node 2
    as_node="node1",
)
result = compiled_graph.invoke(
    # Resume execution by passing None
    None,
    config={"configurable": {"thread_id": "1"}},
    # Stop after node 3 so that node 4 doesn't run
    interrupt_after="node3",
)
assert result["my_key"] == "hello from node3"

```